

UGANDA CHRISTIAN UNIVERSITY

A Center of Excellence in the Heart of Africa

Faculty of Engineering, Design and Technology, Department of Computing and Technology

Course: Intelligent Systems and Robotics

Safety-Jacket Follower Robot: a Fine-tuned YOLOv8 Detector with ROS 2, Gazebo and a Hiwonder MasterPi

Final Project Report

Student 1	Jamugisa Peter Paul (S25M25/008)
Student 2	Ayebare Moses (S25M25/003)
Programme	Master of Science in Computer Science
Date of submission	30th October 2026

1 Introduction

On construction and industrial sites, a small mobile robot that carries tools or materials is most useful if it can follow the worker it is assisting and nobody else. Hi-vis safety jackets are worn on such sites anyway, so they give a natural visual tag. Our project builds a robot that **detects the person wearing a safety jacket and follows them**, keeping a safe gap and speeding up when the person speeds up.

Objectives. (1) Fine-tune an object detector whose single target class is `safety_jacket` (a hi-vis vest with reflective bands) so it works from a low robot camera. (2) Integrate it into ROS 2 with camera input, inference and decision/control nodes. (3) Evaluate the full system in Gazebo in three scenarios, including a challenging and a failure case. (4) Deploy it on a physical Hiwonder MasterPi [7] mecanum robot (Raspberry Pi 4B, arm-mounted USB camera, ultrasonic sensor). The robot must keep about 1.5 m from the worker, search when the jacket is lost, ignore people without jackets and fail safe whenever its inputs are missing or uncertain.

2 Dataset

Our starting model (YOLOv8n) had been trained on 301 eye-level phone photos of one jacket and missed the jacket in 31% of frames from the robot's camera, which sits about 0.25 m above the floor and looks up. We therefore built a larger training set (Table 1):

- **Robot-camera frames.** We recorded sessions from the robot's own camera in the lab. Frames were labelled automatically with Grounding DINO [2] (prompt "safety vest"), filtered by a hi-vis colour check in HSV space (yellow-green fraction $\geq 8\%$) and checked by hand; 57 corrupt frames were discarded.
- **Public data.** 1,500 human-labelled vest photos and 700 photos of people *without* vests from two Roboflow PPE datasets (CC BY 4.0) [3, 4]. The negatives teach the model what is *not* a jacket.

All vest labels were merged into one class, `safety_jacket`, in YOLO bounding-box format. To avoid leakage between near-identical video frames, the robot test frames come from a separate *time block* of the recording, not from a random split, and the public test images are from the datasets' own test splits. Figure 1 shows labelled robot frames.

Table 1: Training data (v2) and the independent test sets (never used for training or model selection).

Source	Images	Labels
Robot-camera frames (used twice to up-weight the deployment view)	328	Grounding DINO + colour filter + human check
Original eye-level phone photos	284	original annotations
Public vest photos	1,500	human (Roboflow, CC BY 4.0)
Public photos of people without vests (negatives)	700	no boxes
Training total / validation	**3,140 / 115**	
Test: robot camera, held-out time block	99	DINO + human check (94 with jacket)
Test: public set A (HB1204)	242	human (186 with vests, 56 without)
Test: public set B (Vincent)	208	human (112 with vests, 96 without)

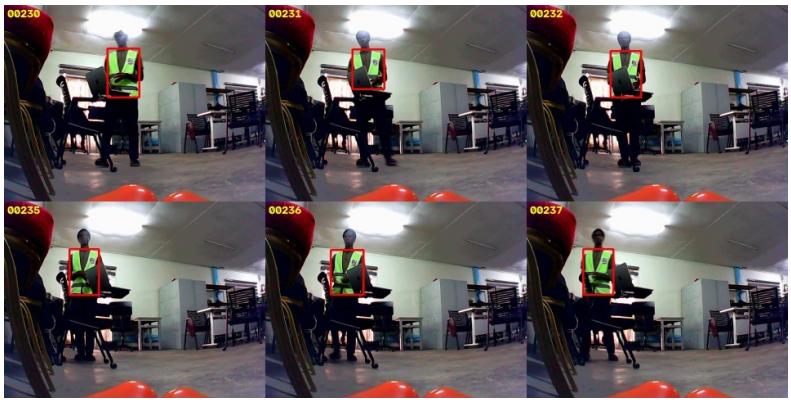


Figure 1: Robot-camera frames with checked `safety_jacket` labels (red boxes). The low viewpoint, chairs and partial views are why the original model struggled.

3 Detector Fine-tuning and Evaluation

We use **YOLOv8n** [1] (3.0 M parameters), a one-stage detector small enough for real-time inference. It was fine-tuned from our original weights at 320 px input with AdamW, a cosine learning-rate schedule and early stopping (best epoch 56 of 76, 31 minutes on an RTX 4050 laptop GPU). Inference on the GPU takes 15–20 ms per frame.

Table 2: Detector results on the independent test sets (320 px). "Found" is the share of frames where the jacket is detected; "false alarms" is the share of no-vest images with any detection.

Test set	Model	Precision	Recall	mAP50	Found	False alarms
Robot camera (99)	Original	0.798	0.777	0.824	69.1%	0%
	Fine-tuned v2	**0.894**	**0.851**	**0.925**	**97.9%**	0%
Public A (242)	Original	0.291	0.193	0.128	38.2%	16.1%
	Fine-tuned v1	0.320	0.159	0.118	48.4%	30.4%
	Fine-tuned v2	**0.864**	**0.738**	**0.811**	**96.8%**	16.1%
Public B (208)	Original	0.501	0.345	0.332	57.1%	21.9%
	Fine-tuned v1	0.509	0.325	0.315	74.1%	33.3%
	Fine-tuned v2	**0.878**	**0.782**	**0.874**	**92.0%**	**14.6%**

Results (Table 2). On the robot camera, mAP50 rose from 0.82 to 0.93 and the jacket is found in 98% of frames instead of 69%. On public photos of other vests, people and places the gain is much larger (mAP50 0.13→0.81 and 0.33→0.87), and false alarms on people without vests fell from 22% to 15% on set B. A first fine-tune (v1) used only robot frames: it detected more jackets but *doubled* false alarms (16→30%). This showed that negative examples are essential, and v2 adds 700 of them. Two cautions apply. The robot test frames come from the same room and day as the robot training frames, and the remaining false alarms are mostly orange or yellow clothing. We handle the latter in the follower (Section 4).

4 ROS 2 System Design

The system is one ROS 2 Jazzy [5] package, `jacket_follower`, running on a laptop (Ubuntu 24.04 in WSL2, RTX 4050). The **same detector and follower nodes** run in Gazebo and on the real robot; only the camera source and the motor output change (Figure 2).

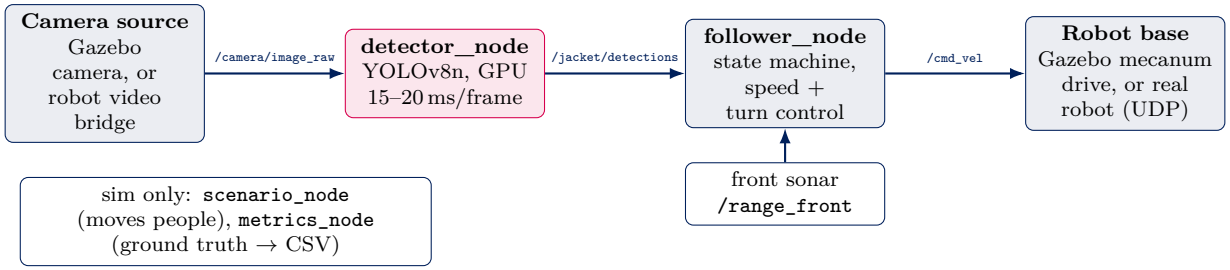


Figure 2: ROS 2 architecture. Detections are `vision_msgs/Detection2DArray`; commands are `geometry_msgs/Twist`.

Camera input and inference. `detector_node` subscribes to `/camera/image_raw` with sensor-data QoS, drops frames while busy so it never builds a backlog, crops the gripper fingertips from the bottom of real-robot images, and publishes boxes, a debug image and its inference time.

Decision and control. `follower_node` runs at 20 Hz with six states: SEARCH, CONFIRMING, FOLLOW, HOLD, TOO_CLOSE and E_STOP. Distance comes from the pinhole model, $d = f \cdot 0.45\,\mathrm{m}/w_{\mathrm{box}}$ (box width), with a second estimate from the box height and, when the target is centred, the sonar. The smallest estimate is used, because each failure mode reads too far: a side-on vest is narrow, and a close vest is cropped by the low camera. The forward speed is

$$v = 0.8\,(d - 1.5) + 0.6\,\dot{d}, \qquad \omega = -1.6\,x_{\mathrm{err}},$$

capped at 0.35 m/s with an acceleration ramp. The $\dot{d}$ term reacts as soon as the gap starts to grow, so the robot **speeds up when the person speeds up**. Here $x_{\mathrm{err}} \in [-1, 1]$ is the horizontal offset of the jacket in the image.

Uncertainty and safety. The robot moves only after 3 consecutive detections. A *new* target must score ≥ 0.70, while the target already being followed is kept down to 0.40, and only near its last bearing. This *lock-on hysteresis* stops hi-vis-coloured clothing (scored ≈ 0.55) from capturing the robot. If the jacket is lost, the robot HOLDs for 1.5 s, then SEARCHes by turning towards where it was last seen. It enters E_STOP (zero velocity) if no detector output arrives for 1 s or the sonar reads under 0.25 m, and it never drives closer than 0.8 m.

5 Gazebo Simulation

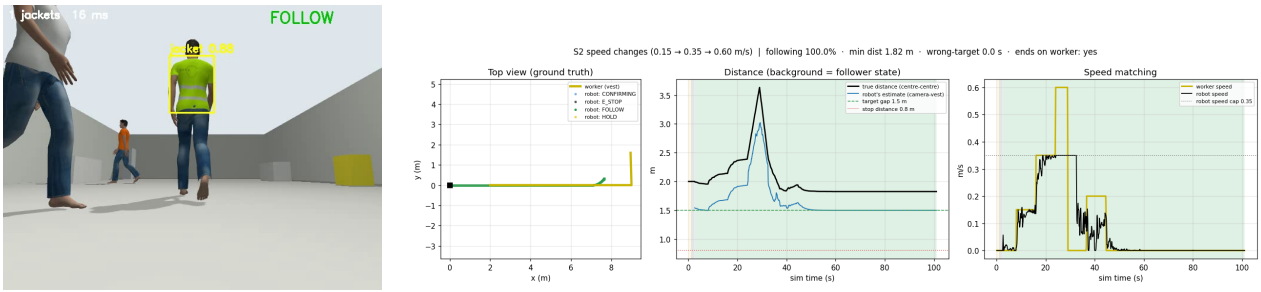
Setup. Gazebo Harmonic [6] with `ros_gz_bridge`. The MasterPi model has mecanum wheels driven by Gazebo's `MecanumDrive` plugin, with diagonal wheel friction modelling the rollers; a camera (640 × 480 at

15 Hz, 89°) on the arm; and a narrow front range sensor standing in for the sonar. The world is a 20 × 10 m hall with desks. The people are textured walking-person models moved by `scenario_node`: the worker in a hi-vis vest with silver bands, a white-shirt bystander and an orange-shirt distractor. Gazebo's odometry gives ground truth, which `metrics_node` logs to CSV at 10 Hz.

Scenarios. S1 normal: the worker walks 4 m at 0.3 m/s, turns 90° twice and walks back. **S2 speed**: the worker walks at 0.15, then 0.35, then 0.60 m/s, stops, turns and walks slowly. **S3 challenging/failure**: the bystander crosses and stands between robot and worker for 20 s while the worker turns and sprints 3.2 m out of view at 0.8 m/s, with the orange shirt in view. We also ran S3 with lock-on hysteresis disabled (acquire threshold 0.40) as an ablation.

Table 3: Gazebo results (ground truth). Gap is the centre-to-centre distance while the worker walks; the closest approach is checked against the 0.8 m stop distance.

Scenario	Following	Wrong person	Gap (m)	Closest	Ends on worker	Speed: worker→robot (m/s)
S1 normal walk	95.9%	0 s	2.07±0.28	1.17 m	yes	0.30→0.19 (incl. turns)
S2 speed changes	100%	0 s	2.27±0.42	1.82 m	yes	0.15→0.13, 0.35→0.32, 0.60→0.35 (cap)
S3 challenge	69.9%	0 s	2.26±0.12	1.93 m	yes	searched 27 s, re-found the worker
S3, no hysteresis	38.7%	**11.2 s**	2.26±0.12	1.95 m	**no**	locked onto the orange shirt



(a) Robot camera in S3. **(b)** S2: top view, true vs. estimated distance, speeds.

Figure 3: Gazebo. (a) The worker is detected (0.88) and followed while the white-shirt and orange-shirt people pass. (b) Ground truth for the speed scenario; background colours are follower states.

Results (Table 3, Figure 3). The robot followed the worker through both turns in S1 and never came closer than 1.17 m. In S2 it matched the worker's speed (0.13 at 0.15 m/s, 0.32 at 0.35 m/s) until its 0.35 m/s cap. At 0.60 m/s the gap grew to 3.6 m, but the worker stayed tracked and the robot closed the gap when they stopped. In S3 the robot lost the worker behind the bystander, searched for 27 s, ignored the orange shirt and found the worker again. **Failure case:** without hysteresis, the robot locked onto the orange shirt and followed the wrong person for 11 s, ending 3.7 m from the worker. This ablation justifies the lock-on rule. Distance estimates were consistent (±0.1 m); their 0.3–0.45 m offset from the table's centre-to-centre ground truth is mostly geometric, since the estimate measures camera to vest surface. Detection took 16.7 ms per frame. Building the simulation also exposed modelling bugs that we fixed: missing roller friction stopped the robot turning in place, and shared mesh textures made every person render in the vest.

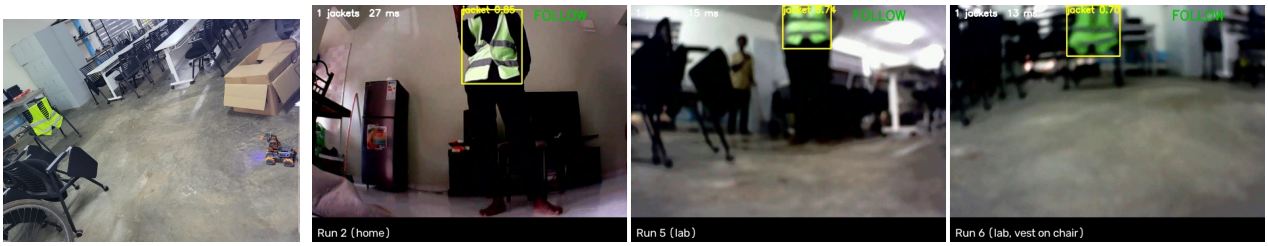
6 Physical Deployment on the MasterPi

The MasterPi runs Raspbian Buster with Python 3.7, which cannot run ROS 2, so we split the system. A small server on the Pi (`masterpi_server.py`) streams the camera over HTTP and drives the mecanum wheels from UDP commands. It has its own 0.5 s watchdog that stops the motors if commands stop, and a hard 60% motor-power cap. On the laptop, `masterpi_bridge` publishes the stream as `/camera/image_raw` and sends `/cmd_vel` to the robot, so the unchanged detector and follower nodes drive the real robot. The arm is set to a camera pose found by image-matching calibration, which also gave the camera's field of view (≈ 71°).

Results. With the wheels disabled, the pipeline ran at 8 frames/s with 20 ms detection, a 10 ms command round trip and about 30 ms camera latency. With the wheels enabled, the robot **searched, detected the jacket (scores 0.70–0.85) and switched to FOLLOW** in three separate runs, at home and in the lab (Figure 4). It turned and drove towards the jacket, and the phone video in our demo shows it moving on its own. However, it held FOLLOW for only up to about 9 s per run before losing the jacket, so steady following was not achieved. The causes were the hardware problems in Table 4.

7 Discussion and Limitations

Fine-tuning with deployment-view data *and* negatives was the key to a usable detector. Ground-truth evaluation in Gazebo showed the follower design works: it follows, matches speed up to the robot's limit, recovers



(a) MasterPi in the lab. **(b)** Robot view: jacket detected, state FOLLOW.

Figure 4: Physical deployment: the real robot locks onto the jacket in three runs (scores 0.85, 0.74, 0.70).

Table 4: Challenges met on the real robot and how we handled them.

Challenge	What happened	Fix / mitigation
Unsafe motor scaling	The SDK's "speed" is PWM duty; our first conversion would have sent 100% for any command	caught by reading the SDK before any wheel moved; 60% hard cap
Motor dead zone	no motion below ∼28–32% power, then sudden fast turns; small corrections overshoot	start-power + speed model; gentler turning on the robot
Battery brown-outs	the two 18650 cells sagged from 8.0 to 6.2 V under load; the Pi rebooted mid-run	motor power scaled by battery voltage; slow, one-servo-at-a-time arm moves
Runaway after reboot	the motor board kept its last speed when the Pi rebooted, so the robot spun until stopped remotely	a boot service zeroes all motors at start-up (the watchdog lives on the Pi, so it cannot cover a reboot)
Camera blur and dim light	out of focus; 1.5 frames/s at 640 × 480; a jacket on a chair at 3 m was not detected at all	manual focus, MJPG mode (8.5 frames/s), camera tilt; lock-on threshold 0.50 on the robot
Frame rate while driving	2–3 frames/s under motor load on a weak battery, so the robot reacts late	2 s watchdog, stop-and-look search (turn, pause, look)
Wi-Fi drops	the laptop silently left the robot's hotspot (no internet)	auto-reconnect; the robot stops itself when commands stop

from occlusion and, thanks to hysteresis, does not switch to a look-alike. The real robot showed what simulation hides. Power, light and actuators limited performance more than the algorithms did, and several safety mechanisms, such as the boot-time motor stop, were only discovered on hardware. Limitations: the robot test set is from one room and day; the 0.35 m/s cap means the robot falls behind faster walkers; a person closer than about 1 m is outside the low camera's view; detection range drops in dim light; and the real-robot results are a few short runs, not a measured benchmark.

Future work. Fresh batteries and a repeat of the following test, with ground truth from a tape-marked course; a test set from a new location and lighting; pan-servo tracking so the camera keeps the jacket in view while the chassis turns; a camera with better low-light exposure; and running the detector on the robot itself (ONNX or TFLite) to remove the Wi-Fi dependency.

8 Conclusion

We fine-tuned a YOLOv8n safety-jacket detector (robot-camera mAP50 0.82→0.93, jacket found in 98% of frames) and built a ROS 2 follower with distance-based speed matching, search and fail-safe states. In Gazebo the robot followed the worker in all three scenarios, and our ablation showed that lock-on hysteresis prevents following the wrong person. On the physical MasterPi the full pipeline is deployed and the robot detects and locks onto the jacket. Steady real-world following is limited by the battery, camera and motors, and these are our next steps.

9 Individual Contributions

Jamugisa Peter Paul (S25M25/008)	Detector: robot-camera data collection, Grounding DINO auto-labelling and checking, dataset building, fine-tuning v1/v2 and independent evaluation. Physical deployment: robot server, ROS 2 bridge, camera and motor calibration, safety fixes and live robot tests.
Ayebare Moses (S25M25/003)	ROS 2 package: detector, follower, bridge, scenario and metrics nodes, launch files and parameters. Gazebo: MasterPi model, hall world, people models, the three scenarios, ground-truth metrics and simulation evaluation.

References

[1] G. Jocher, A. Chaurasia and J. Qiu, *Ultralytics YOLOv8*, 2023. https://github.com/ultralytics/ultralytics

[2] S. Liu *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," arXiv:2303.05499, 2023.

[3] HB1204, *PPE_Detection* dataset (Roboflow export, CC BY 4.0), Hugging Face, 2024.

[4] V. Gourbin, *ppe-detection* dataset (Roboflow export, CC BY 4.0), Hugging Face, 2024.

[5] S. Macenski *et al.*, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, 7(66), 2022.

[6] Open Robotics, *Gazebo Harmonic*, https://gazebosim.org, 2023.

[7] Hiwonder, *MasterPi documentation*, https://docs.hiwonder.com/projects/MasterPi.